Name: Kande Madithya
Student Reference Number: 10965440

IN PARTNERSHIP WITH PLYMOUTH UNIVERSITY

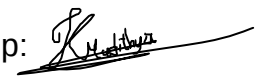
Module Code: PUSL2019	Module Name: Information Management and Retrieval
Coursework Title: Utility Management System (UMS) – Database Design and Implementation	
Deadline Date: 12/01/2026	Member of staff responsible for coursework: Mr.S.Nasiketha
Programme: BSc Hons Software Engineering Plymouth University	

Please note that University Academic Regulations are available under Rules and Regulations on the University website www.plymouth.ac.uk/studenthandbook.

Group work: please list all names of all participants formally associated with this work and state whether the work was undertaken alone or as part of a team. Please note you may be required to identify individual responsibility for component parts.

1.Kande Madithya
2.Kaveesha Jayaweera
3.Viraj Vithanaarachchi
4.Herath Herath
5.Bogaha Nethma
6. Aldora Weerarathna
7. Sakun Arachchi
8. Imaya Lamahewa
9. Kaveesha Kulathunga
10. Warshakoon Warshakoon

We confirm that we have read and understood the Plymouth University regulations relating to Assessment Offences and that we are aware of the possible penalties for any breach of these regulations. We confirm that this is the independent work of the group.

Signed on behalf of the group: 

Individual assignment: ***I confirm that I have read and understood the Plymouth University regulations relating to Assessment Offences and that I am aware of the possible penalties for any breach of these regulations. I confirm that this is my own independent work.***

Signed:

Use of translation software: failure to declare that translation software or a similar writing aid has been used will be treated as an assessment offence. I *have used/not used translation software. If used, please state name of software……
   Overall mark _____% Assessors Initials _____ Date_____

*Please delete as appropriateSci/ps/d:/students/cwkfrontcover/2013/14

1. Introduction

The Utility Management System (UMS) developed in this project as requested for this modules assignment is a database-driven web application using Microsoft SQL Server Management Studio is designed to manage electricity, water, and gas services in a structured, secure, and automated manner. Utility service providers typically deal with large volumes of customers, meters, readings, bills, and payments. When handling such task manually it becomes a hassle and would cause major inaccuracies, most times leading to delays, billing errors, and poor customer experiences.

The aim of this project was to design and implement a centralized database system that handles customer management, meter tracking, billing, payments, penalties, and reporting within a single platform. The system was built around a relational database named **Logindata**, which ensures data consistency, reduces redundancy, and supports scalability.

The UMS supports multiple user roles including customers, meter_readers, cashiers, administrators, and managers, each with distinct permissions. Customers can view their usage, bills, and make payments through their dashboard. Meter readers submit readings, cashiers approve payments, managers oversee reports and billing cycles, and administrators maintain full system control.

Additional features were given to this system when building this project in hopes of making the user experience better and fair for all. The profile management, light and dark mode, penalties being applied for overdue bills, and a reporting module enhance usability and operational efficiency.

This report documents the database design, implementation, normalization, integrity mechanisms, and advanced database features developed for the Utility Management System, demonstrating how the system satisfies both functional requirements and academic best practices.

2. System Overview

The Utility Management System is integrated with all utility-related data within a centralized relational database called Logindata. Instead of maintaining separate systems for electricity, water, and gas services, UMS adopts a unified data model that allows multiple utility types to be handled consistently.

Core system functionalities include:

- User registration and role management
- Allocation of meters to customers
- Entry of periodic meter readings
- Automated bill generation based on tariffs
- Payment processing and approval
- Application of penalties for overdue bills
- Internal reporting and communication

The database structure supports operational processes such as utility consumption tracking, billing accuracy, payment auditing, and management reporting. By automating these activities, UMS reduces human error and ensures reliable, real-time access to critical business information.

3. Users and Roles

The system defines five primary roles, each with controlled access to the database:

3.1 Customer

Customers access a personalized dashboard where they can:

- View meter readings
- Send reports to managers
- Select the Utilities that are being used by the customer
- Make payments
- Edit account details and profile picture
- Switch between light and dark modes

3.2 Meter Reader

Meter readers are responsible for:

- Submitting periodic meter readings
- Updating usage data for assigned customers
  Their role ensures accurate data collection at the operational level.

3.3 Cashier

Cashiers manage the financial workflow by:

- Approving and recording payments
- Viewing customer payment histories
- Ensuring balances are updated correctly

3.4 Manager

Managers perform supervisory functions such as:

- Reviewing system reports and customer complaints
- Resetting monthly billing processes
- Modifying user roles

3.5 Administrator

Administrators have full control over the system:

- Managing all user accounts
- Editing sensitive fields such as UserID, email, location, and passwords
- Overseeing system configuration and integrity

4. Utilities Supported

The UMS is designed to manage three core utilities within a single framework:

4.1 Electricity

Electricity usage is recorded in kilowatt-hours (kWh). Bills are calculated using tariff slabs stored in the **Tariffs** table, supporting fixed charges, variable rates, and category-based pricing.

4.2 Water

Water consumption is measured in cubic meters (m^3). Meter readings determine usage, and billing incorporates unit-based pricing and service charges.

4.3 Gas

Gas usage is also metered and billed according to defined tariff structures. The system supports differentiated pricing based on consumption ranges and customer categories.

5. Database Design

5.1 ER Model

The database design follows an Entity Relationship (ER) approach to model entities, relationships, and constraints.

Entities Identified:

- Users
- CustomerMeters
- MeterReadings
- Bills
- BillDetails
- Payments
- Penalties
- Tariffs
- Reports
- ReportMessages

Key Relationships:

- One User → Many CustomerMeters
- One CustomerMeter → Many MeterReadings
- One User → Many Bills
- One Bill → Many BillDetails
- One Bill → Many Payments
- One Bill → Many Penalties
- One Report → Many ReportMessages

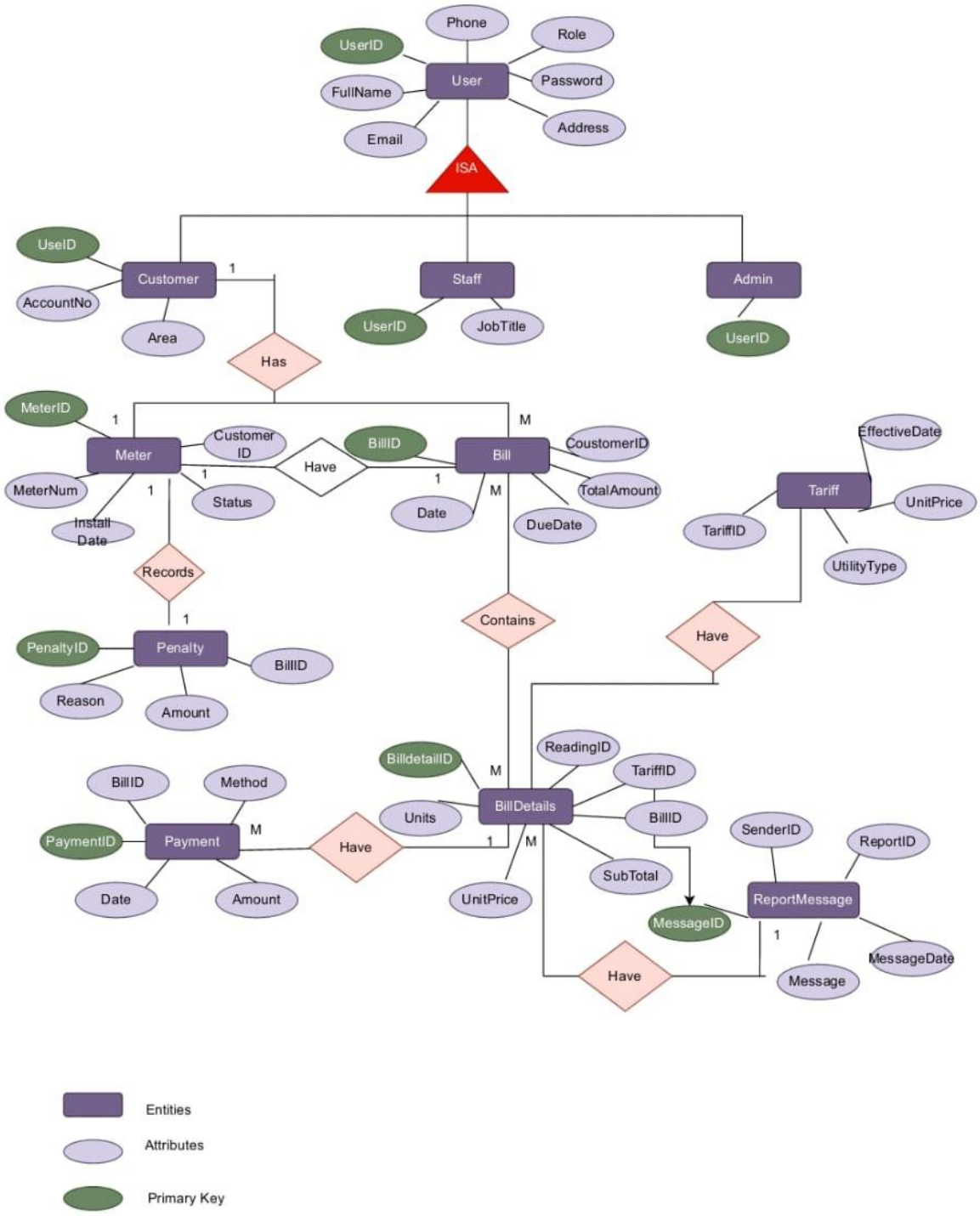


Figure 01 Entity Relationship diagram for the project

5.2 Relational Mapping (ER → Tables)

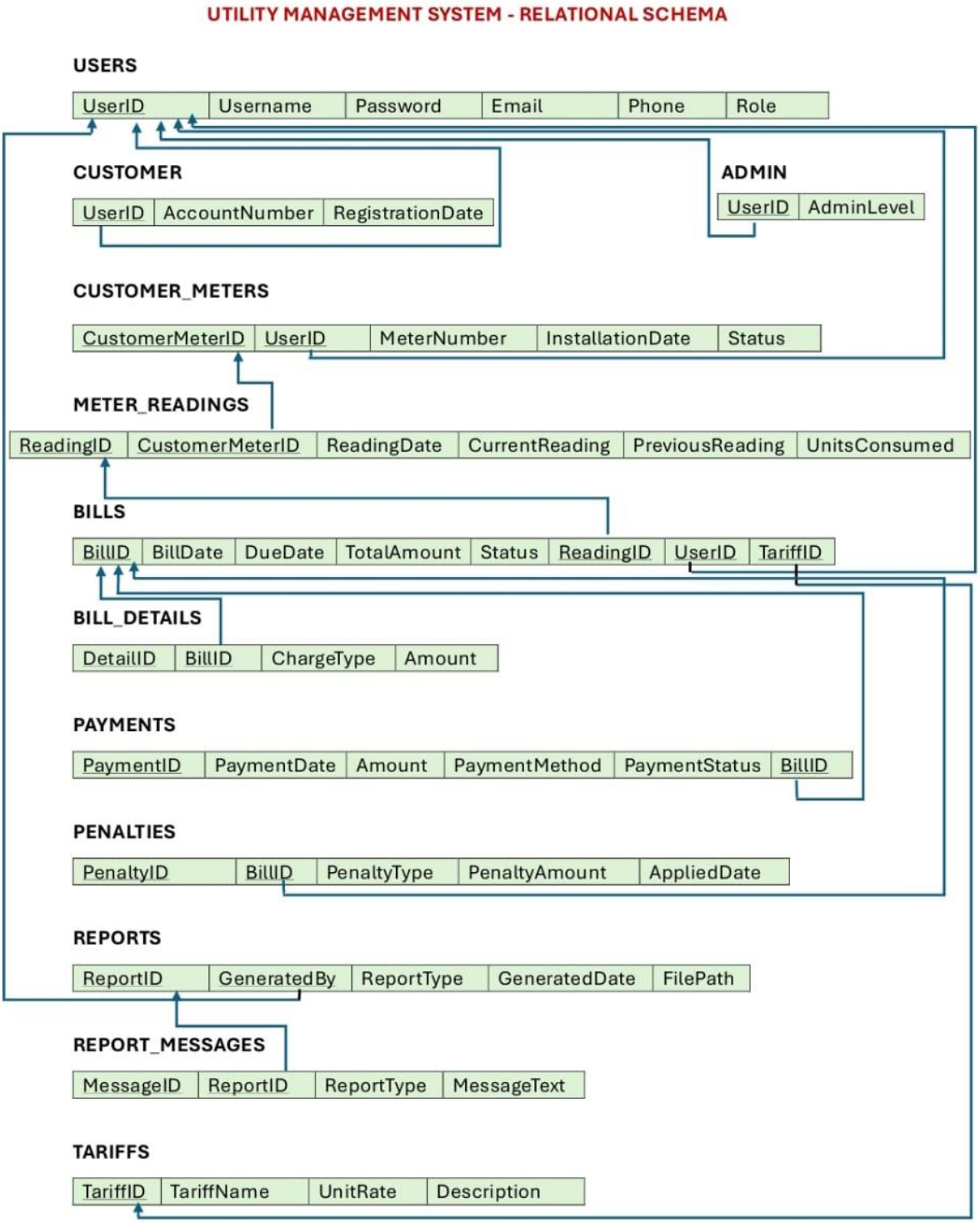


Figure 02 the relational mapping diagram of the system

5.3 Design Assumptions

The following assumptions on different users and databases guided the database design:

- A user can have multiple utility meters (Electricity,gas,water).
- Each meter belongs to only one user and each meter got a unique meter ID.
- A bill may include multiple utilities, stored in BillDetails.
- Every payment must be associated with a valid bill.
- Penalties shall apply only to overdue bills.
- Only authorized roles (cashier, admin, manager) may modify financial records.
- A user who is logging in is automatically logged in as a customer until the Manager or an Administrator promotes that user into a different role.
- Just paying the amount in the bill does not make the user a paid user, the user must wait till the cashier accepts the payment.
- Required customer/user information when signing in are name, Email, Location(Address) and a password. Charged may vary with rate, but older bill retain previous charges.
- Only one valid reading per meter is recorded for each billing cycle.
- Partial payments are allowed but the bill status changes into paid only when the payment is done in full.
- Each user is assigned exactly one role, not multiple roles.
- Customers only have read only access to billing, usage and payment data without the ability to modify.
- Historical data is never deleted.
- Each utility type is billed using unit based consumption multiplied by a rate defined in the tariffs table.

5.4 Normalization

To make sure the Utility Management System database is efficient, reliable, and easy to manage, normalization principles were applied during the design process. The database was carefully structured to avoid repeating the same data, reduce the risk of errors when updating records, and keep clear relationships between different entities such as users, meters, bills, and payments. Each table was broken down based on functional dependencies and organized to meet Third Normal Form (3NF). This means that every piece of information in a table depends only on the primary key and not on other non-key fields. As a result, the system becomes more accurate, easier to maintain, and capable of scaling as the number of customers, meter readings, and transactions grows over time. The diagrams below show how the data was normalized and how the final relational structure was achieved.

1.Normal Form (1NF)

BillID	CustomerName	Address	MeterNumbers	MeterReadings	TarriffRates	PaymentIDs
B001	Kaveesha	Matara	M01	125	15	P01

2.Normal Form (2NF)

Customer Table

CustomerID	CustomerName	Address
C001	Kaveesha	Matara

Meter Table

MeterNumber	CustomerID
M01	C001

Biliing(2NF)

BillID	MeterNumber	MeterReading	TariffRate
B001	M01	125	15

3.Normal Form(3NF)

Customer Table

CustomerID	CustomerName	Address
C001	Kaveesha	Matara

Meter Table

MeterNumber	CustomerID
M01	C001

Tariff Table

TariffID	Rate
T01	15

Billing(3NF)

BillID	MeterNumber	TariffID	MeterReading
B001	M01	T01	125

Payment Table

PaymentID	BillID	Amount	PaymentDate
P01	B001	2750	2026-01-06

Figure 03 Normalization of the tables in our UMS

5.5 Data Dictionary for the tables

Table: Users

Field Name	Data Type	Constraints	Description
UserID	INT	Primary Key, Identity	Unique identifier for each user in the system.
Username	VAR-CHAR(100)	NOT NULL, UNIQUE	Login name used by the user.
Email	VAR-CHAR(150)	NOT NULL, UNIQUE	User's registered email address.
Password	VAR-CHAR(255)	NOT NULL	Encrypted password for authentication.
Location	VAR-CHAR(150)	NULL	Address or region associated with the user.
Role	VAR-CHAR(50)	NOT NULL, DEFAULT 'customer'	Defines user role (customer, admin, manager, cashier, meter_reader).
ProfileI-mage	VAR-CHAR(255)	NOT NULL, DEFAULT 'default.png'	Stores filename of the user's profile picture.

Table: CustomerMeters

Field Name	Data Type	Constraints	Description
CustomerMe-terID	INT	Primary Key, Identity	Unique identifier for each meter as-signed to a customer.
UserID	INT	Foreign Key → Users(UserID), NOT NULL	Links meter to a specific customer.
MeterType	VAR-CHAR(50)	NOT NULL	Type of utility (electricity, water, gas).
MeterID	VAR-CHAR(100)	NOT NULL, UNIQUE	Physical meter identification number.
IsActive	BIT	NOT NULL, DEFAULT 1	Indicates whether the meter is currently active.

Table: MeterReadings

Field Name	Data Type	Constraints	Description
ReadingID	INT	Primary Key, Identity	Unique identifier for each meter reading.
CustomerMeterID	INT	Foreign Key → CustomerMeters(CustomerMeterID), NOT NULL	Links reading to a specific meter.
MeterReaderID	INT	Foreign Key → Users(UserID), NOT NULL	Identifies the meter reader who recorded the reading.
PreviousReading	DECIMAL(10,2)	NOT NULL	Last recorded meter value.
CurrentReading	DECIMAL(10,2)	NOT NULL	Newly recorded meter value.
ReadingMonth	INT	NOT NULL	Month when the reading was taken.
ReadingYear	INT	NOT NULL	Year when the reading was taken.
ReadingDate	DATE	NOT NULL	Exact date of the meter reading.

Table: Bills

Field Name	Data Type	Constraints	Description
BillID	INT	Primary Key, Identity	Unique identifier for each bill.
UserID	INT	Foreign Key → Users(UserID), NOT NULL	Identifies the customer being billed.
BillingMonth	INT	NOT NULL	Month of the billing period.
BillingYear	INT	NOT NULL	Year of the billing period.
TotalAmount	DECIMAL(10,2)	NOT NULL	Total payable amount for the bill.
DueDate	DATE	NOT NULL	Deadline for bill payment.
Status	VARCHAR(50)	NOT NULL	Payment status (Pending, Paid, Overdue).
CreatedAt	DATETIME	DEFAULT GETDATE()	Date and time when the bill was generated.

Table : BillDetails

Field Name	Data Type	Constraints	Description
BillDetai-lID	INT	Primary Key, Identity	Unique identifier for bill line item.
BillID	INT	Foreign Key → Bills(BillID), NOT NULL	Links details to a specific bill.
UtilityType	VARCHAR(50)	NOT NULL	Utility type (electricity, water, gas).
UnitsUsed	DECI-MAL(10,2)	NOT NULL	Number of units consumed.
Rate	DECI-MAL(10,2)	NOT NULL	Price per unit applied.
Amount	DECI-MAL(10,2)	NOT NULL	Calculated cost for this utility.

Table: Payments

Field Name	Data Type	Constraints	Description
PaymentID	INT	Primary Key, Identity	Unique identifier for each payment.
BillID	INT	Foreign Key → Bills(BillID), NOT NULL	Identifies the bill being paid.
PaidAmount	DECI-MAL(10,2)	NOT NULL	Amount paid by the customer.
PaymentDate	DATETIME	DEFAULT GETDATE()	Date and time of payment.
Payment-Method	VARCHAR(50)	NOT NULL	Method of payment (cash, card, online).
CashierID	INT	Foreign Key → Users(UserID), NULL	Cashier who processed the payment.
Status	VARCHAR(50)	NOT NULL	Payment status (Approved, Pending, Failed).

Table: Penalties

Field Name	Data Type	Constraints	Description
PenaltyID	INT	Primary Key, Identity	Unique identifier for penalty rec-ord.
BillID	INT	Foreign Key → Bills(BillID), NOT NULL	Bill to which the penalty applies.
PenaltyRate	DECIMAL(5,2)	NOT NULL	Percentage rate applied as pen-alty.
Penal-yAmount	DECI-MAL(10,2)	NOT NULL	Calculated penalty amount.
AppliedDate	DATE	NOT NULL	Date when penalty was applied.

Table: Tariffs

Field Name	Data Type	Constraints	Description
TariffID	INT	Primary Key, Identity	Unique identifier for tariff rule.
UtilityType	VARCHAR(50)	NOT NULL	Utility category (electricity, water, gas).
UnitFrom	DECIMAL(10,2)	NOT NULL	Minimum unit range for tariff.
UnitTo	DECIMAL(10,2)	NOT NULL	Maximum unit range for tariff.
PricePerUnit	DECIMAL(10,2)	NOT NULL	Cost applied per unit in this range.
EffectiveFrom	DATE	NOT NULL	Date when tariff becomes active.
IsActive	BIT	NOT NULL	Indicates whether tariff is currently valid.

Table: Reports

Field Name	Data Type	Constraints	Description
ReportID	INT	Primary Key, Identity	Unique identifier for a report or complaint.
SenderID	INT	Foreign Key → Users(UserID), NOT NULL	User who submitted the report.
Subject	VARCHAR(200)	NOT NULL	Brief description of the report.
Status	VARCHAR(50)	NOT NULL	Current state (Open, In Progress, Closed).
CreatedAt	DATETIME	DEFAULT GETDATE()	Timestamp of report submission.

Table: ReportMessages

Field Name	Data Type	Constraints	Description
MessageID	INT	Primary Key, Identity	Unique identifier for each message.
ReportID	INT	Foreign Key → Reports(ReportID), NOT NULL	Links message to a report thread.
SenderID	INT	Foreign Key → Users(UserID), NOT NULL	User who sent the message.
Message	TEXT	NOT NULL	Content of the message.
CreatedAt	DATETIME	DEFAULT GETDATE()	Time message was sent.
IsRead	BIT	DEFAULT 0	Indicates whether message has been read.

6. Database Implementation

6.1 Table Creation

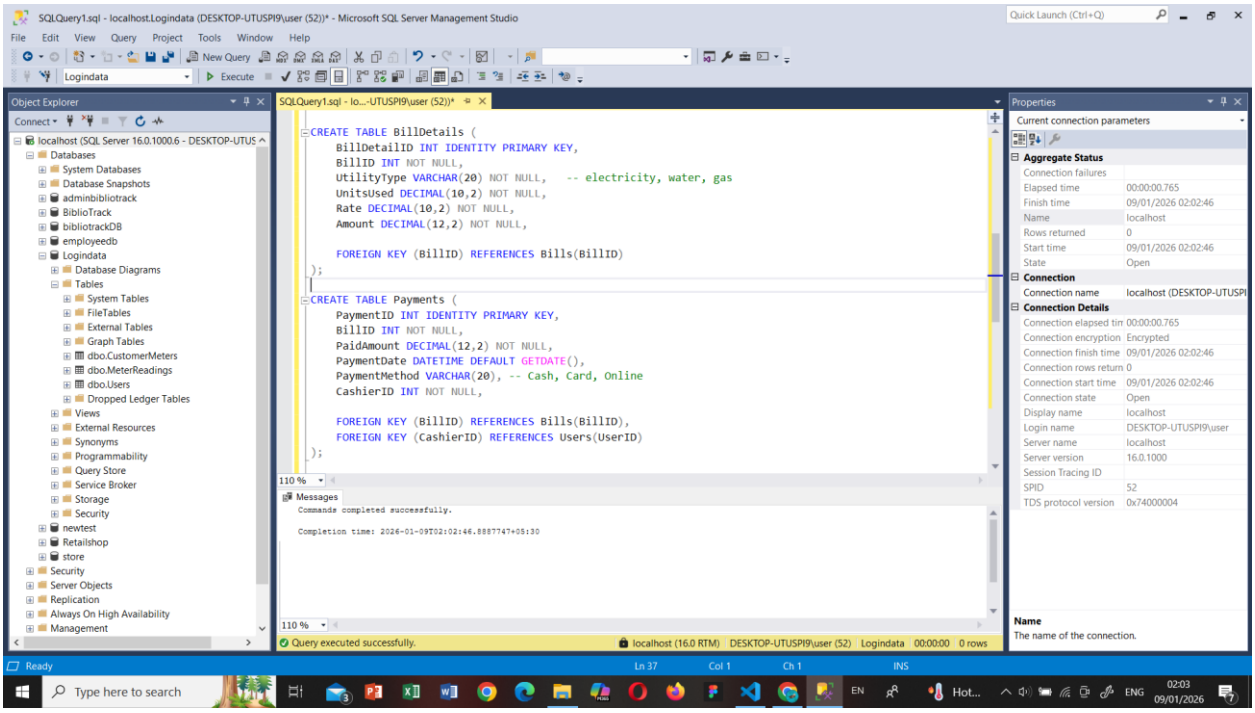


Figure 04 illustration of the queries for BillDetails and Payments tables

```
/****** Object:  Table [dbo].[Tariffs]    Script Date: 12/01/2026 03:59:04 ******/
SET ANSI_NULLS ON
GO

SET QUOTED_IDENTIFIER ON
GO

CREATE TABLE [dbo].[Tariffs](
    [TariffID] [int] IDENTITY(1,1) NOT NULL,
    [UtilityType] [varchar](20) NOT NULL,
    [UnitFrom] [decimal](10, 2) NOT NULL,
    [UnitTo] [decimal](10, 2) NOT NULL,
    [PricePerUnit] [decimal](10, 2) NOT NULL,
    [EffectiveFrom] [date] NOT NULL,
    [IsActive] [bit] NULL,
PRIMARY KEY CLUSTERED
(
    [TariffID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENT
) ON [PRIMARY]
GO

ALTER TABLE [dbo].[Tariffs] ADD  DEFAULT ((1)) FOR [IsActive]
GO
```

Figure 05 Illustration on the queries of Tariffs table

```
/****** Object:  Table [dbo].[Reports]    Script Date: 12/01/2026 04:05:04 ******/
SET ANSI_NULLS ON
GO

SET QUOTED_IDENTIFIER ON
GO

CREATE TABLE [dbo].[Reports](
    [ReportID] [int] IDENTITY(1,1) NOT NULL,
    [SenderID] [int] NOT NULL,
    [Subject] [nvarchar](255) NOT NULL,
    [Status] [nvarchar](20) NULL,
    [CreatedAt] [datetime] NULL,
PRIMARY KEY CLUSTERED
(
    [ReportID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTI
) ON [PRIMARY]
GO

ALTER TABLE [dbo].[Reports] ADD  DEFAULT ('Unread') FOR [Status]
GO

ALTER TABLE [dbo].[Reports] ADD  DEFAULT (getdate()) FOR [CreatedAt]
GO

ALTER TABLE [dbo].[Reports]  WITH CHECK ADD FOREIGN KEY([SenderID])
REFERENCES [dbo].[Users] ([UserID])
GO
```

Figure 06 illustration of the queries for Reports table

```
USE [Logindata]
GO

/****** Object:  Table [dbo].[Users]    Script Date: 12/01/2026 07:13:33 ******/
SET ANSI_NULLS ON
GO

SET QUOTED_IDENTIFIER ON
GO

CREATE TABLE [dbo].[Users](
    [UserID] [int] IDENTITY(1,1) NOT NULL,
    [Username] [nvarchar](50) NOT NULL,
    [Email] [nvarchar](100) NOT NULL,
    [Password] [nvarchar](255) NOT NULL,
    [Location] [nvarchar](100) NULL,
    [Role] [varchar](20) NOT NULL,
    [ProfileImage] [varchar](255) NULL,
```

Figure 07 illustration shows the queries of the Users table

```
USE [Logindata]
GO

/****** Object:  Table [dbo].[ReportMessages]    Script Date: 12/01/2026 07:18:56 ******/
SET ANSI_NULLS ON
GO

SET QUOTED_IDENTIFIER ON
GO

CREATE TABLE [dbo].[ReportMessages](
    [MessageID] [int] IDENTITY(1,1) NOT NULL,
    [ReportID] [int] NOT NULL,
    [SenderID] [int] NOT NULL,
    [Message] [nvarchar](max) NOT NULL,
    [CreatedAt] [datetime] NULL,
    [IsRead] [bit] NULL,
```

Figure 08 illustration shows the queries of the ReportMessages table

```
USE [Logindata]
GO

/****** Object:  Table [dbo].[MeterReadings]    Script Date: 12/01/2026 07:30:55 ******/
SET ANSI_NULLS ON
GO

SET QUOTED_IDENTIFIER ON
GO

CREATE TABLE [dbo].[MeterReadings](
    [ReadingID] [int] IDENTITY(1,1) NOT NULL,
    [CustomerMeterID] [int] NOT NULL,
    [MeterReaderID] [int] NOT NULL,
    [PreviousReading] [decimal](10, 2) NOT NULL,
    [CurrentReading] [decimal](10, 2) NOT NULL,
    [ReadingMonth] [int] NOT NULL,
    [ReadingYear] [int] NOT NULL,
    [ReadingDate] [datetime] NULL,
```

Figure 09 illustration shows the queries of the MeterReadings table

```
USE [Logindata]
GO

/****** Object:  Table [dbo].[Penalties]    Script Date: 12/01/2026 07:35:16 ******/
SET ANSI_NULLS ON
GO

SET QUOTED_IDENTIFIER ON
GO

CREATE TABLE [dbo].[Penalties](
    [PenaltyID] [int] IDENTITY(1,1) NOT NULL,
    [BillID] [int] NOT NULL,
    [PenaltyRate] [decimal](5, 2) NULL,
    [PenaltyAmount] [decimal](12, 2) NOT NULL,
    [AppliedDate] [datetime] NULL,
```

Figure 10 illustration shows the queries of the Penalties table

```
USE [Logindata]
GO

/****** Object:  Table [dbo].[CustomerMeters]    Script Date: 12/01/2026 08:02:05 ******/
SET ANSI_NULLS ON
GO

SET QUOTED_IDENTIFIER ON
GO

CREATE TABLE [dbo].[CustomerMeters](
    [CustomerMeterID] [int] IDENTITY(1,1) NOT NULL,
    [UserID] [int] NOT NULL,
    [MeterType] [varchar](20) NOT NULL,
    [MeterID] [varchar](50) NOT NULL,
    [IsActive] [bit] NULL,
```

Figure 11 illustration shows the queries of the CustomerMeters table

6.2 Sample Data

```
SELECT TOP (1000) [UserID]
      ,[Username]
      ,[Email]
      ,[Password]
      ,[Location]
      ,[Role]
      ,[ProfileImage]
  FROM [Logindata].[dbo].[Users]
```

0 %

Results | Messages

UserID	Username	Email	Password	Location	Role	ProfileImage
1	kaveesha	kaveesha@gmail.com	$2y$10$e878r4kAUnVBQ.OqD/LJIOcQp/Ly1bTxE2G0f2jeLVoN16v...	matara	meter_reader	pf_694647b55844d.jpeg
2	admin	admin@gmail.com	$2y$10$zbs0rvggNi5bWXPd.Mday.DB6l110ZhtLPwJy.2HG8EB58z...	nsbm	customer	pf_694647600d37a.jpeg
3	jithmi	jithmi@gmail.com	$2y$10$sarLh.uacEgsQBBBT.17m.K2uuG75ywxg77sCSu8PzUYR...	tangalle	customer	NULL
4	pathaan	pathaan@gmail.com	$2y$10$Q9VSwbaWLIIHFJEJRRaKLugAUXoJK2OI0Z/2YOycu1TQf...	india	customer	NULL
5	Jkmam	jk@gmail.com	$2y$10$s31Tir4jYGw1EQN3WhX2t.nywDuGF46DA4WjJtAg3U6ISL...	nsbm	customer	pf_694645bf494a7.jpeg
6	UI/UX Upeksha	upeksha@gmail.com	$2y$10$APuG5Rwh2zZiGepQgdRK.OZRJBu4vZx8VKuC3TE.CAfe...	monaragala	cashier	pf_69462615e1510.jpeg
7	nadishan	nadishan@gmail.com	$2y$10$T2BiN.NnN.L44lLoVpuwR.k6sxippRgS3lYUaYd3pGHR0d...	mathale	customer	default.png
8	sakun	sakun@gmail.com	$2y$10$IAG.BOhsH0LM14oxlvoiourD60eCOpQelhQcJQuSp5qmJ...	kaluthara	customer	default.png
9	amadi	amadi@gmail.com	$2y$10$dLolhVpneIICAi5oFGNcC.NZb3ogI01Dzln4faSdYiMSgWF...	gampaha	customer	default.png
10	kavesha	kavesha@gmail.com	$2y$10$yNmE954.IP0213XGPsa2cONJhWGF2srBVCNBwEzNmP...	gampaha	manager	default.png
11	kavindu	kavindu@gmail.com	$2y$10$x4Vcw5JM3c4IhN8Dlvrtju7BICqRXLaZOVZbIYwM4zHL.5...	homagama	admin	default.png
12	testing	test@gmail.com	$2y$10$gHx.jEw.pqCVeP8JLZjcrueILsP8aC53j3htlKj.qxJmNYhdv...	nsbm	customer	default.png

Figure 12 the sample data collected in the Users table

110 %

Results | Messages

	CustomerMeterID	UserID	MeterType	MeterID	IsActive
1	1	5	electricity	EM-482739	1
2	2	5	water	WM-552901	1
3	3	5	gas	GM-442190	1
4	4	6	electricity	EM-343241	1
5	5	6	water	WM-984365	1
6	6	6	gas	GM-904358	1
7	7	8	electricity	EM-343222	1
8	8	8	water	WM-984374	1
9	9	8	gas	GM-904397	1
10	10	2	electricity	EM-343345	1
11	11	2	water	WM-984983	1
12	12	2	gas	GM-904745	1
13	13	9	electricity	EM-343222	1
14	14	9	water	WM-984055	1
15	15	9	gas	GM-904475	1

Figure 13 illustrates the collected data from CustomerMeter table

Results Messages

	TariffID	UtilityType	UnitFrom	UnitTo	PricePerUnit	EffectiveFrom	IsActive
1	2	Electricity	0.00	60.00	8.00	2024-01-01	1
2	3	Electricity	61.00	120.00	12.00	2024-01-01	1
3	4	Electricity	121.00	999999.00	18.00	2024-01-01	1
4	5	Water	0.00	20.00	5.00	2024-01-01	1
5	6	Water	21.00	50.00	10.00	2024-01-01	1
6	7	Water	51.00	999999.00	15.00	2024-01-01	1
7	8	Gas	0.00	999999.00	25.00	2024-01-01	1

Figure 14 illustrates the collected data from Tariffs table

	ReadingID	CustomerMeterID	MeterReaderID	PreviousReading	CurrentReading	ReadingMonth	ReadingYear	ReadingDate
1	1	1	1	0.00	350.00	12	2025	2025-12-20 08:31:47.000
2	2	2	1	0.00	42.00	12	2025	2025-12-20 08:39:43.000
3	3	3	1	0.00	13.00	12	2025	2025-12-20 08:39:49.000
4	4	1	1	0.00	120.00	1	2025	2025-01-31 00:00:00.000
5	5	1	1	120.00	145.00	2	2025	2025-02-28 00:00:00.000
6	6	1	1	145.00	170.00	3	2025	2025-03-31 00:00:00.000
7	7	1	1	170.00	195.00	4	2025	2025-04-30 00:00:00.000
8	8	1	1	195.00	220.00	5	2025	2025-05-31 00:00:00.000
9	9	1	1	220.00	245.00	6	2025	2025-06-30 00:00:00.000
10	10	1	1	245.00	270.00	7	2025	2025-07-31 00:00:00.000
11	11	1	1	270.00	295.00	8	2025	2025-08-31 00:00:00.000
12	12	1	1	295.00	320.00	9	2025	2025-09-30 00:00:00.000
13	13	1	1	320.00	340.00	10	2025	2025-10-31 00:00:00.000
14	14	1	1	340.00	350.00	11	2025	2025-11-30 00:00:00.000
15	15	2	1	0.00	20.00	1	2025	2025-01-31 00:00:00.000
16	16	2	1	20.00	24.00	2	2025	2025-02-28 00:00:00.000
17	17	2	1	24.00	27.00	3	2025	2025-03-31 00:00:00.000
18	18	2	1	27.00	30.00	4	2025	2025-04-30 00:00:00.000
19	19	2	1	30.00	33.00	5	2025	2025-05-31 00:00:00.000
20	20	2	1	33.00	36.00	6	2025	2025-06-30 00:00:00.000
21	21	2	1	36.00	38.00	7	2025	2025-07-31 00:00:00.000
22	22	2	1	38.00	40.00	8	2025	2025-08-31 00:00:00.000
23	23	2	1	40.00	41.00	9	2025	2025-09-30 00:00:00.000

Figure 15 illustrates the collected data from MeterReadings table

Results Messages

	BillDetailID	BillID	UtilityType	UnitsUsed	Rate	Amount
1	1	16	electricity	70.00	12.00	840.00
2	2	16	water	3.00	10.00	30.00
3	3	16	gas	7.00	25.00	175.00
4	4	19	electricity	260.00	12.00	3120.00
5	5	19	water	29.00	10.00	290.00
6	6	19	gas	15.00	25.00	375.00
7	7	13	electricity	400.00	12.00	4800.00
8	8	13	water	200.00	10.00	2000.00
9	9	13	gas	15.00	25.00	375.00
10	10	20	electricity	250.00	12.00	3000.00
11	11	20	water	200.00	10.00	2000.00
12	12	20	gas	29.00	25.00	725.00

Figure 16 illustrates the collected data from BillDetails table

Results Messages

	BillID	UserID	BillingMonth	BillingYear	TotalAmount	DueDate	Status	CreatedAt
1	13	2	1	2026	7175.00	2026-01-24	Paid	2026-01-10 17:48:40.053
2	14	3	1	2026	0.00	2026-01-24	UNPAID	2026-01-10 17:48:40.167
3	15	4	1	2026	0.00	2026-01-24	UNPAID	2026-01-10 17:48:40.170
4	16	5	1	2026	1045.00	2026-01-24	Paid	2026-01-10 17:48:40.173
5	17	6	1	2026	0.00	2026-01-24	UNPAID	2026-01-10 17:48:40.373
6	18	7	1	2026	0.00	2026-01-24	UNPAID	2026-01-10 17:48:40.377
7	19	8	1	2026	3785.00	2026-01-24	UNPAID	2026-01-10 20:15:58.977
8	20	9	1	2026	5725.00	2026-01-24	Paid	2026-01-10 20:52:22.477

Figure 17 illustrates the collected data from Bill table

Results Messages

	PaymentID	BillID	PaidAmount	PaymentDate	PaymentMethod	CashierID	Status
1	2	16	1045.00	2026-01-10 19:47:19.793	Online	6	Approved
2	3	19	3700.00	2026-01-10 20:32:35.907	Cash	6	Approved
3	4	13	7175.00	2026-01-10 20:39:06.730	Cash	6	Approved
4	5	20	5725.00	2026-01-10 20:52:33.243	Cash	6	Approved

Figure 18 illustrates the collected data from Payments table

Results Messages

	MessageID	ReportID	SenderID	Message	CreatedAt	IsRead
1	1	5	5	testing to see if this works	2026-01-10 23:44:41.243	1
2	2	5	10	yes it works well	2026-01-10 23:54:34.920	1
3	3	5	10	yeah	2026-01-11 00:00:06.057	1
4	4	6	5	sending this to see iv everything is working as ...	2026-01-11 00:00:49.837	1
5	5	6	10	it is working fine	2026-01-11 00:04:02.873	1
6	6	7	5	testing to see if the new strcutre is better	2026-01-11 00:39:25.380	1
7	7	7	10	yes it seems to be better	2026-01-11 00:39:50.370	1

Figure 19 illustrates the collected data from ReportMessages table

Results Messages

	ReportID	SenderID	Subject	Status	CreatedAt
1	5	5	testing	Unread	2026-01-10 23:44:41.220
2	6	5	testing2	Unread	2026-01-11 00:00:49.710
3	7	5	testing3	Replied	2026-01-11 00:39:25.300

Figure 20 illustrates the collected data from Reports table

7.1 Triggers

One of the triggers were created on the **Payments** table to automatically update the status of a bill. Whenever a payment is recorded and approved, the trigger recalculates the total amount paid for that bill. If the full balance has been settled, the corresponding record in the **Bills** table is automatically updated to “Paid”. This ensures that bill statuses always reflect the actual payment situation without requiring manual updates.

A second trigger was implemented on the **Bills** table to handle overdue payments. If a bill remains unpaid after its due date, the trigger automatically inserts a record into the **Penalties** table. This allows late-payment charges to be applied consistently and fairly, without requiring staff intervention.

Overall, the use of triggers strengthens the reliability of the system by enforcing key business rules directly at the database level. This approach improves data integrity, reduces the risk of inconsistencies, and demonstrates the use of more advanced database features within the Utility Management System.

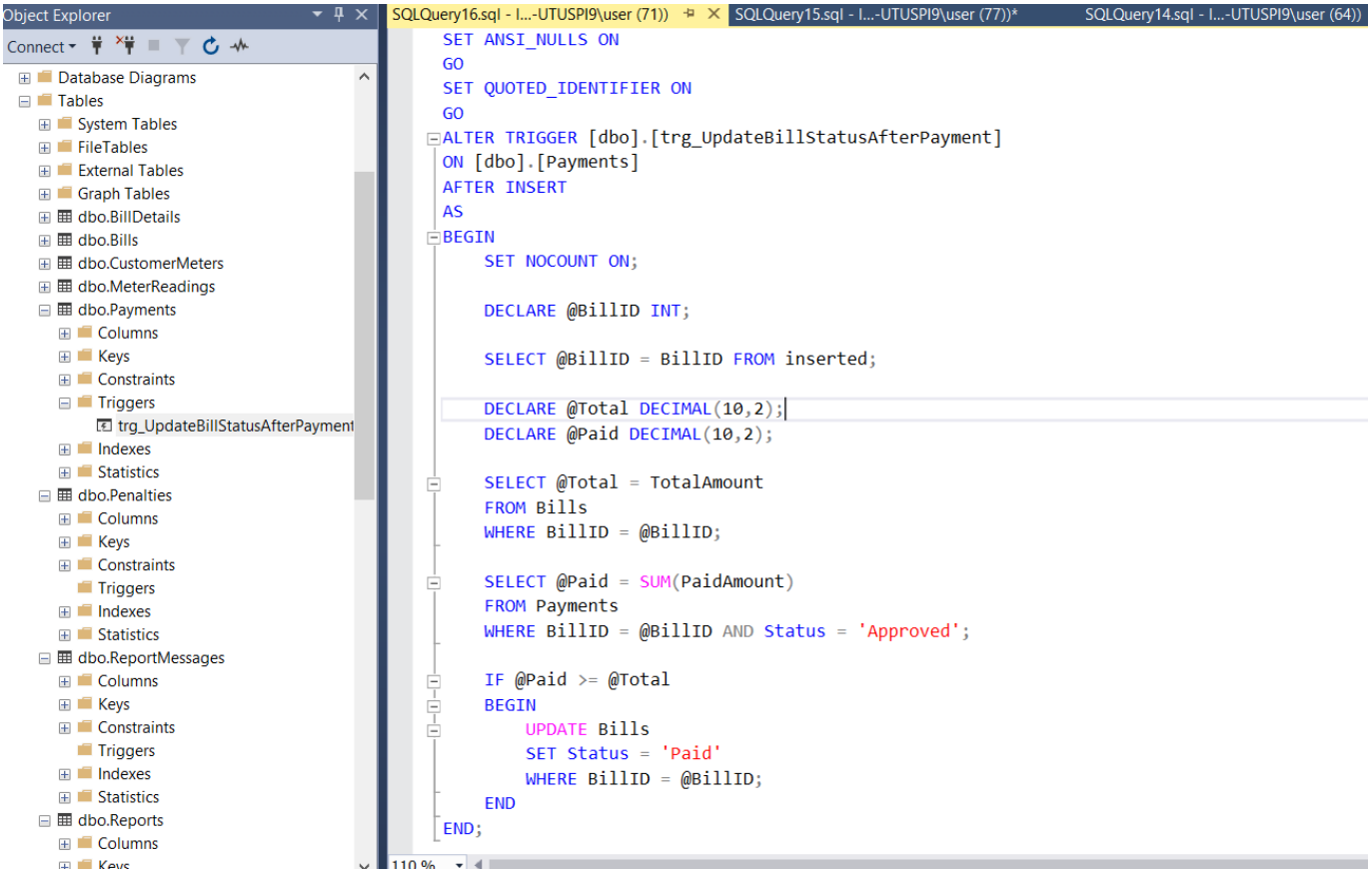


Figure 21 trigger to update the bill status right after a payment

Figure 22 trigger to update the system so that penalties would be applied to overdue bills

7.2 User Defined Functions

```sql
CREATE FUNCTION dbo.fn_CalculateBillTotal (@BillID INT)
RETURNS DECIMAL(10,2)
AS
BEGIN
    DECLARE @Total DECIMAL(10,2);

    SELECT @Total = SUM(Amount)
    FROM BillDetails
    WHERE BillID = @BillID;

    RETURN ISNULL(@Total, 0);
END;
```

```
Messages
Commands completed successfully.

Completion time: 2026-01-12T12:07:29.7077763+05:30
```

Figure 23 Encapsulates billing logic into a reusable database function.

```sql
CREATE FUNCTION dbo.fn_CalculateLatePenalty
(
    @BillID INT
)
RETURNS DECIMAL(10,2)
AS
BEGIN
    DECLARE @Penalty DECIMAL(10,2);
    DECLARE @DueDate DATE;
    DECLARE @TotalAmount DECIMAL(10,2);
    DECLARE @DaysLate INT;

    SELECT @DueDate = DueDate, @TotalAmount = TotalAmount
    FROM Bills
    WHERE BillID = @BillID;

    SET @DaysLate = DATEDIFF(DAY, @DueDate, GETDATE());

    IF @DaysLate > 0
        SET @Penalty = @TotalAmount * 0.01 * @DaysLate; -- 1% per day
    ELSE
        SET @Penalty = 0;

    RETURN @Penalty;
END;
```

```
Messages
Commands completed successfully.

Completion time: 2026-01-12T12:12:13.2507251+05:30
```

Figure 24 Encodes business rules for late fee calculation.

7.3 Views

```
SELECT TOP (1000) [PaymentID]
      ,[BillID]
      ,[PaidAmount]
      ,[PaymentDate]
      ,[PaymentMethod]
      ,[CashierID]
      ,[Status]
  FROM [Logindata].[dbo].[Payments]

  CREATE VIEW UnpaidBills AS
SELECT BillID, UserID, TotalAmount, DueDate, Status
FROM Bills
WHERE Status = 'Unpaid';
```

110 %

Messages

```
Commands completed successfully.

Completion time: 2026-01-12T06:31:38.8800224+05:30
```

Figure 25 view to see all unpaid bills

```
CREATE VIEW MonthlyRevenue AS
SELECT BillingMonth, BillingYear, SUM(TotalAmount) AS TotalRevenue
FROM Bills
WHERE Status = 'Paid'
GROUP BY BillingMonth, BillingYear;
```

10 %

Messages

```
Commands completed successfully.

Completion time: 2026-01-12T06:57:38.5808064+05:30
```

Figure 26 view to see a monthly revenue summary

Stored Procedures

```sql
CREATE PROCEDURE sp_GenerateBillForCustomer
    @UserID INT,
    @Month INT,
    @Year INT
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @BillID INT;

    INSERT INTO Bills (UserID, BillingMonth, BillingYear, TotalAmount, DueDate, Status, CreatedAt)
    VALUES (@UserID, @Month, @Year, 0, DATEADD(DAY, 14, GETDATE()), 'Pending', GETDATE());

    SET @BillID = SCOPE_IDENTITY();

    INSERT INTO BillDetails (BillID, UtilityType, UnitsUsed, Rate, Amount)
    SELECT
        @BillID,
        cm.MeterType,
        (mr.CurrentReading - mr.PreviousReading) AS UnitsUsed,
        t.PricePerUnit,
        (mr.CurrentReading - mr.PreviousReading) * t.PricePerUnit
    FROM CustomerMeters cm
    JOIN MeterReadings mr ON cm.CustomerMeterID = mr.CustomerMeterID
    JOIN Tariffs t ON t.UtilityType = cm.MeterType AND t.IsActive = 1
    WHERE cm.UserID = @UserID
      AND mr.ReadingMonth = @Month
      AND mr.ReadingYear = @Year;
```

```
110 %
Messages
Commands completed successfully.

Completion time: 2026-01-12T12:14:27.7342841+05:30
```

Figure 27 Automates bill generation and ensures consistency using functions

```sql
CREATE PROCEDURE sp_ListDefaulters
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        u.UserID,
        u.Username,
        u.Email,
        b.BillID,
        b.TotalAmount,
        b.DueDate,
        b.Status
    FROM Bills b
    JOIN Users u ON b.UserID = u.UserID
    WHERE b.Status <> 'Paid'
      AND b.DueDate < CAST(GETDATE() AS DATE);
END;
```

```
) %
Messages
Commands completed successfully.

Completion time: 2026-01-12T12:17:20.9240183+05:30
```

Figure 28 Identifies overdue accounts for follow-up and reporting.

8. Data Integrity and Security

The UMS enforces integrity using primary and foreign keys, NOT NULL and UNIQUE constraints, CHECK conditions, and triggers. Transactions ensure that billing and payment updates are either fully committed or rolled back in case of error.

Security is achieved through role-based access control (RBAC), authentication, and restricted customer access to internal data. Administrative privileges are limited to authorized staff. Regular backups and validation routines further protect data reliability.

---

9. Evaluation

Strengths

- We got a centralized data model.
- Accurate and transparent billing.
- Role-based access is there making it a clean and a more professional site.
- Automated billings and such in the system would reduce errors and inaccuracies.
- The whole system got a scalable design that we can improve for the future implementations

Weaknesses

- we haven’t directly integrated banks into the system yet and so far it is just taking in values directly from the user
- Meter readings are all manually entered instead of it being automatic through the means of some IOT device
- Real time analysis is lacking due to it being not integrated to any iot devices.

---

10. Future Enhancements

- For self services it is possible to add an online self servicing system
- Developing a mobile app for meter readers
- Smart meter integration
- Adding real time payment gateways into this system
- We can use AI to give predictions by going through their utility consumption history.

---

11. Conclusion

The Utility Management System (UMS) developed in this project demonstrates a well-structured, scalable, and reliable database-driven solution for managing multiple utility services within a single integrated platform. By applying core database principles such as normalization, referential integrity, and role-based access control, the system ensures that data remains accurate, consistent, and secure across all operations. These design choices significantly reduce redundancy, prevent data anomalies, and support efficient handling of large volumes of customer, meter, billing, and payment records.

The automation of key processes such as bill generation, payment tracking, penalty application, and reporting greatly enhances operational efficiency and minimizes the likelihood of human error. Features such as triggers, structured relationships between tables, and clearly defined user roles (customer, meter reader, cashier, manager, and administrator) enable the system to enforce business rules directly at the database level while maintaining accountability and transparency. As a result, both day-to-day operations and long-term decision-making processes are supported by accurate, up-to-date information.

Beyond its current implementation, the UMS provides a strong technical foundation for future expansion. The system is designed in a way that allows for the seamless integration of additional technologies, such as online customer portals, mobile applications for meter readers, smart meter connectivity, and real-time payment gateways. These enhancements would further improve accessibility, customer experience, and overall service efficiency.

In conclusion, the Utility Management System not only fulfills the functional requirements of managing electricity, water, and gas services, but also demonstrates the application of sound database engineering principles in a real-world context. The project highlights how thoughtful system design, automation, and security considerations can transform traditionally manual and error-prone utility management processes into a streamlined, accurate, and future-ready digital solution.

Link for the Presentation Video,

https://liveplymouthac-my.sharepoint.com/:v:/g/personal/10965559_students_plymouth_ac_uk/IQB3QrF7RhszTK7T2YcxTJXmASLoMCJjL16krVY6cM8dpK4?e=qZPykR

Link for the Project source Folder,

https://drive.google.com/drive/folders/15nmc-gGAtXrnco2s5Jt_CYO6VtsqdAtJt?usp=sharing

12. Contribution matrix

Name	StudentID	Role	Key Contributions
Kande Madithya	10965440	Lead Developer / Database Designer	Designed overall database schema, created core tables, implemented billing logic, designed the UI for the system, implmeneted the authentication system. was involved in all of the development parts throughout the project.
Kaveesha Jayaweera	10965528	Support Developer	Assisted in debugging, UI polishing, payment module testing, and system optimization. Was involved in utility usage side.
Viraj Vithanaarachchi	10965627	Database Engineer	Created normalization, foreign key constraints, assisted with triggers and stored procedures, tested queries. Was involved in designing the database as well.
Herath Herath	10965516	Frontend Developer	Designed customer dashboard UI, implemented light/dark mode toggle, improved usability and styling. Was involved in the login signin part
Kaveesha Kulathunga	10965552	ER & Schema Designer/support developer	Created ER diagrams, relational schema diagrams, and prepared screenshots for documentation. Was involved in developing the manager side of the project
Imaya Lamahewa	10965559	Testing & QA/ support developer	Tested system workflows, validated billing accuracy, checked role permissions and payment processes. Was involved in developing the admin side.
Warshakoon Warshakoon	10965630	Support Developer	Was involved in developing the payment side of the system and designing the tables necessary for that
Bogaha Nethma	10965571	ER & Schema Designer/support developer	Created ER diagrams, relational schema diagrams, and prepared screenshots for documentation. Was involved in developing the customer repots side of the project.
Aldora Weerarathna	10965314	Data Analyst	Designed reporting queries, tested defaulter reports, assisted in analytics section. Was involved in developing the chart for the dashboard
Sakun Arachchi	10965445	Support Developer	Was involved in developing the payment side of the system and designing the tables necessary for that